

Отчёт об исследовании и тестировании продукта: Gachapets

Параметр	Значение
Объект аудита	Веб-приложение Gachapets (JS SPA + FastAPI + SQLite/PostgreSQL)
Методология	Black-Box / Gray-Box (Postman, без UI(GUI)), White-Box (Code Audit), DoS / Boundary Testing
Критический вердикт	Выявлены критические архитектурные уязвимости

1. Введение и цель исследования

Паспорт исследования

- Объект тестирования:** Архитектура взаимодействия веб-приложения (клиент на JS + API на FastAPI + база данных SQLite/PostgreSQL).
- Методология:** Комбинированное тестирование бизнес-логики (Black-Box / Gray-Box) с использованием Postman, инспекция исходного кода (White-Box), симуляция аномальных действий пользователя и проверка границ устойчивости API.
- Цель:** Оценить корректность распределения бизнес-логики между клиентом и сервером, выявить векторы манипуляции результатами гачи и протестировать защищённость базы данных от нелегитимных записей и спам-атак.

2. Результаты тестирования: Выявленные уязвимости и аномалии

Дефект № 1: Доверие клиенту в критической игровой логике (Client-Side Authority)

Что происходит:

Весь генератор случайных чисел (`Math.random()`), расчёт редкости (`getRoll`), пул персонажей и счетчик гарантированного выпадения (*pity*) находятся на стороне браузера в файлах `gacha.js` и `storage.js`.

Результат тестирования в Postman:

Эндпоинт POST /api/log принимает от клиента уже готовый результат: cat_title и rarity. Бэкенд проверяет лишь то, входит ли присланный кот в белый список (ALLOWED_CATS).

POST /api/log HTTP/1.1
Host: api.gachapets.local
Content-Type: application/json
{
"uuid": "ryan_gosling_2049",
"id-session": "ryan_gosling_ya_metrika"
"cat_title": "Властелин Прайда",
"rarity": "legendary"
}

Итог: Пользователь может отправить 100 запросов подряд с телом {"uuid": "ryan_gosling_2049", "id_session": "ryan_gosling_ya_metrika", "cat_title": "Властелин Прайда", "rarity": "legendary"}, и сервер послушно запишет в базу данных 100 легендарных котов подряд. Система гаранта (Pity System) и вероятности (5%, 15%, 25%, 55%) фактически не защищены.

Дефект № 2: Подделка идентичности и засорение базы данных (Identity Spoofing)

❑ HIGH | Отсутствие серверной аутентификации сессий Класс уязвимости: Broken Authentication / Resource Exhaustion

Уровень критичности: Высокий (High)

Что происходит:

Идентификатор пользователя (user_uuid) генерируется в браузере с помощью Math.random() и хранится в localStorage. В коде функции get_or_create_user: если сервер видит новый user_uuid, он автоматически создаёт нового пользователя в таблице users.

Итог: Отправляя запросы с новыми случайными строками в поле user_uuid, атакующий может создать в таблице users миллионы фейковых профилей, исчерпав дисковое пространство и память базы данных (Database Exhaustion / Resource Starvation).

Дефект № 3: Уязвимость к количественным атакам (Missing Rate Limiting & DoS)

❑ HIGH | Ненадлежащий контроль генерации частых запросов.

Уровень критичности: Высокий (High)

Что происходит:

На эндпоинтах /api/log и /api/event отсутствует контроль частоты запросов (Rate

Limiting). Клиентский таймер ожидания анимации (800 мс) реализован исключительно в `app.js` через `setTimeout`.

Результат тестирования в Postman:

Используя Postman Collection Runner или скрипт, можно за секунды отправить десятки тысяч валидных запросов. Сервер выполнит синхронные транзакции в SQLite/PostgreSQL, что приведет к:

- Блокировке базы данных (особенно актуально для SQLite с блокировкой файла на запись).
- Искажению всей продуктовой аналитики и Яндекс.Метрики.
- Перегрузке пула соединений SQLAlchemy.

3. Матрица рисков

Уязвимость	Вектор атаки	Влияние на бизнес/продукт	Критичность
Генерация кота на фронте	Прямой вызов API с подделкой JSON	Нарушение игровой экономики, обесценивание механики редкости	Критический (Critical)
Автосоздание юзеров по UUID	Генерация случайных ID в цикле	Засорение БД, отказ дисковой подсистемы	Высокий (High)
Отсутствие Rate Limiting	Быстрый флуд запросами через Postman	Падение бэкенда (DoS), искажение метрик	Высокий (High)
Хранение Pity в LocalStorage	Редактирование через DevTools	Любой игрок может выставить себе 19 круток и выбить легендарку	Средний (Medium)

4. Концептуальное решение: Куда переносить логику и как её разделить?



Фундаментальное правило разработки сетевых приложений «Клиент скомпрометирован по умолчанию. Клиенту нельзя доверять ничего, кроме нажатия кнопок».

Что должно остаться на фронтенде:

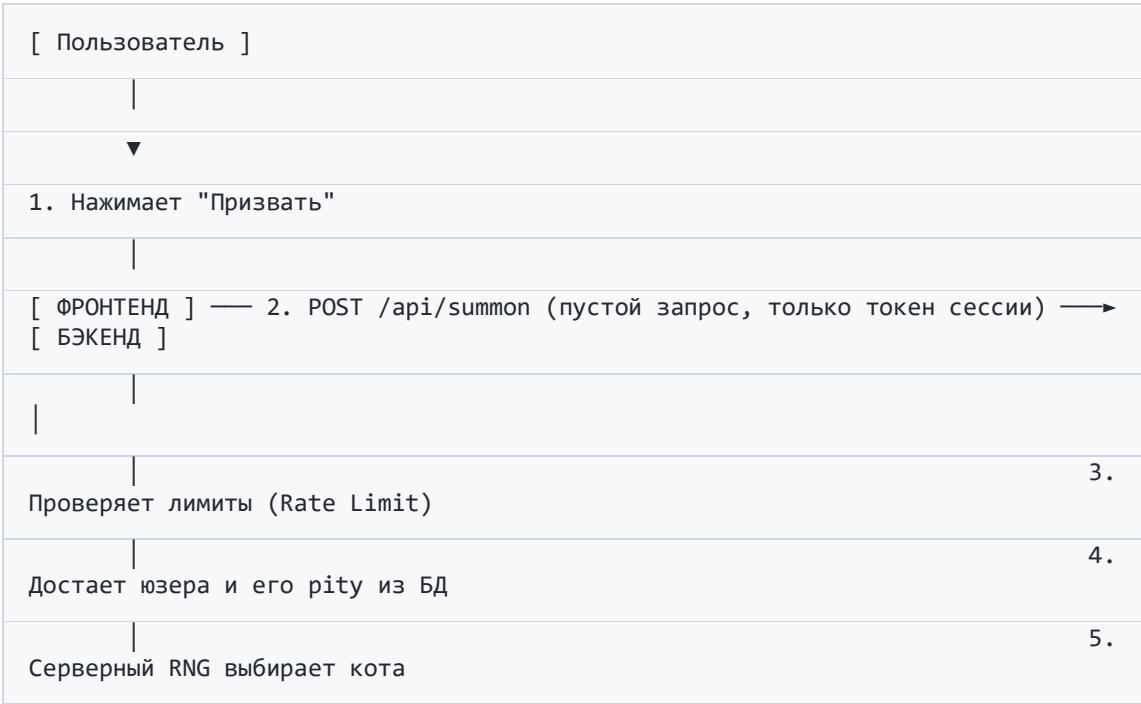
- **Интерфейс и визуализация:** отрисовка карточки кота, анимация крутки, звуки, всплывающие окна, модальки с описанием.
- **Тайминги анимации:** анимация ожидания результата.
- **Сбор локальных пользовательских событий:** переключение вкладок, клик по интерфейсу (для аналитики UI).

Что ОБЯЗАТЕЛЬНО переносится на бэкенд:

- **Справочник персонажей и шансы дропа:** список котов, их веса, вероятности (5%, 15%, 25%, 55%).
- **Генератор случайных чисел (RNG):** выбор кота должен происходить исключительно в функции на Python.
- **Состояние гаранта (pity counter):** сервер читает текущий счётчик круток игрока из БД, рассчитывает результат, сбрасывает или инкрементирует счётчик и сам сохраняет его в БД.
- **Учёт попыток и тайм-ауты (кулдауны):** сервер проверяет, прошло ли достаточно времени с прошлой крутки.

5. Как связать фронтенд и бэкенд: Новый жизненный цикл запроса

Вместо схемы, где клиент сообщает серверу о выигрыше, внедряется схема **запроса на действие**:



Обновляет pity и сохраняет лог в БД	6.
[ФРОНТЕНД] ← 7. Ответ 200 OK { cat_title, rarity, tag, new_pity, img_url }	
8. Запускает анимацию и выводит полученного кота	

Что меняется на уровне API:

1. **Удалить эндпоинт `/api/log`** в текущем виде. Клиент не должен отправлять `cat_title` и `rarity`.
2. **Создать транзакционный эндпоинт `POST /api/summon`**: клиент шлет запрос на призыв, сервер сам решает, кого выдать, и возвращает данные для отображения вместе с обновленными счетчиками.

6. Методы защиты от спама и количественных атак

1. **Серверный Rate Limiting (Ограничение частоты):**
 - По IP-адресу: не более 30 запросов в минуту. При превышении — мгновенный HTTP 429 Too Many Requests без нагрузки на БД.
 - По идентификатору сессии: ограничение действует на аккаунт (например, 1 крутка в 2 секунды), даже если атакующий меняет прокси.
2. **Игровой кулдаун (Cooldown / Timestamp check):**
 - В БД у пользователя фиксируется поле `last_summon_at`. Если разница с текущим временем меньше 1.5–2 секунд, запрос отклоняется: *«Слишком быстро, котики ещё отдыхают»*.
3. **Переход от фиктивного UUID к безопасным сессиям:**
 - Инициализация на сервере. При первом входе сервер выдает подписанный токен или `HttpOnly, Secure` cookie. Автосоздание профилей по клиентскому заголовку ликвидируется.
4. **Атомарность операций (Защита от Race Conditions):**
 - Чтение гаранта, выбор кота и обновление счетчика выполняются в рамках одной атомарной транзакции БД с блокировкой строки пользователя (`FOR UPDATE`).
5. **Поведенческий анализ и капча:**
 - При длительных монотонных скриптовых запросах система временно блокирует призыв и требует капчу (Cloudflare Turnstile).

7. Резюме и план рефакторинга

- **Этап 1:** Перенести массивы данных с котами (`personalities.js`) и логику вероятностей (`gacha.js`) на Python-бэкенд.

- **Этап 2:** Заменить клиентский эндпоинт логирования `/api/log` на эндпоинт действия `/api/summon`.
- **Этап 3:** Перенести хранение `pity` и статистики из `localStorage` в таблицу БД.
- **Этап 4:** Внедрить Rate Limiting и проверку интервалов между крутками на стороне FastAPI.
- **Этап 5:** Перевести аутентификацию пользователей на серверные токены/cookie.

8. Детальная декомпозиция этапов

Этап 1. Перенос пула персонажей и генератора случайных чисел (RNG) на бэкенд

Суть этапа Лишение фронтенда роли «источника правды» об игровом балансе. Математика дропа и список редкостей полностью изолируются внутри серверного контура.

Что создаст:

- **Серверный конфигурактор баланса (Single Source of Truth):** реестр персонажей на бэкенде. Фронтенд больше не скачивает `personalities.js`.
- **Изолированный модуль серверного RNG:** розыгрыш на базе криптографически стойких генераторов Python вместо предсказуемого `Math.random()`.
- **Скрытие контента:** секретные и ивентовые коты недоступны через DevTools до их выпадения.

Что даст:

- Защиту от реверс-инжиниринга вероятностей.
- Управление экономикой «на лету» без обновления фронтенда и сброса кэша пользователей.
- Ликвидацию подделки результата через изменение памяти браузера.

Этап 2. Замена пассивного логирования (`/api/log`) на активное действие (`/api/summon`)

Суть этапа Переход от архитектуры «Уведомление о свершившемся факте» к архитектуре «Запрос на авторизованное действие» (Action/RPC-модель).

Что создаст:

- **Эндпоинт POST `/api/summon`:** не принимает имя кота и редкость, только контекст пользователя.
- **Стандартизированный ответ:** сервер сам отдает персонажа, картинку и обновленные счетчики.
- **Удаление уязвимого `/api/log`:** ликвидируется шлюз ввода произвольных данных.

Что даст:

- 100% ликвидацию атаки через подмену JSON в Postman.
- Превращение фронтенда в «чистый терминал» (Dumb UI).
- Гарантированную синхронизацию данных между визуалом и базой.

Этап 3. Перенос счётчика гаранта (pity) и статистики из localStorage в базу данных

Суть этапа Привязка состояния игрового прогресса к учетной записи на сервере вместо локального хранилища браузера.

Что создаст:

- **Расширение схемы таблицы пользователей в БД:** поля `pity_counter`, `total_summons`, `legendary_count`.
- **Транзакционный конечный автомат (State Machine):** чтение pity → расчёт → сброс/инкремент → коммит в рамках одной транзакции.

Что даст:

- Защиту от накрутки гаранта через DevTools (`localStorage.setItem('pityCounter', 19)`).
- Персистентность данных при смене браузера, устройства или чистке кэша.
- Истинность продуктовой аналитики.

Этап 4. Внедрение Rate Limiting и игровых кулдаунов

Суть этапа Построение двухслойного защитного барьера от высокочастотных автоматизированных запросов и скриптового флуда.

Что создаст:

- **Middleware ограничения частоты:** отсечение флуда по IP и сессии с отдачей HTTP 429.
- **Серверный кулдаун:** проверка `last_summon_at` (блокировка запросов быстрее 1.5–2 секунд).

Что даст:

- Защиту от DoS-атак и зависания БД (критично для блокировок SQLite).
- Чистоту аналитики от мусорных скриптовых кликов.
- Принудительное соблюдение ритма игры (защита от автокликеров).

Этап 5. Переход с фиктивного UUID на управляемую сервером аутентификацию

Суть этапа Ликвидация бесконтрольного создания аккаунтов на основе строки из браузера и внедрение криптографически подписанных сессий.

Что создаст:

- **Серверный менеджер сессий:** контролируемая первичная выдача идентификатора.
- **Безопасные HttpOnly Cookie:** клиентские скрипты не имеют прямого доступа к сессии.
- **Удаление get_or_create_user:** запрет на автосоздание записей по произвольному ID.

Что даст:

- Защиту от исчерпания ресурсов БД миллионами фейковых аккаунтов.
- Защиту от спуфинга и кражи чужих профилей.
- Возможность серверной блокировки и модерации пользователей.

Сводная синергия всех 5 этапов

Если внедрить только:	Что произойдет? (Слабое звено)
Только Этапы 1 и 2 (Логика на бэке)	Результат кота подделать нельзя, но злоумышленник сможет спамить миллионами случайных аккаунтов и быстро перегрузить базу через Postman.
Только Этапы 3 и 4 (Pity в БД + Rate limit)	Нагрузку на сервер снизим, но если генерация останется на фронте, клиент продолжит сам присылать себе нужных котов в обход шансов.

 **Все 5 этапов в комплексе: Система полностью закрыта:** Клиент ничего не решает, база данных защищена от мусора и перегрузок, экономика контролируется сервером, а спам отсекается на ранних этапах.